

HOSPITAL APPOINTMENT MANAGEMENT

PROGRAM OUTPUT SCREENSHOT

Hospital Appointment Management

File Help

HOSPITAL APPOINTMENT MANAGEMENT

Appointment Details

Patient Name: Rahul Sharma

Doctor: Dr. Priya

Date (DD-MM-YYYY): 18-08-2026

Time: 10:30 AM

Book Update Cancel Clear

Patient	Doctor	Date	Time	Status
Rahul Sharma	Dr. Priya	18-08-2026	10:30 AM	Confirmed

SOURCE CODE

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.ArrayList;
import java.util.List;

public class HospitalAppointmentGUI extends JFrame {
    private JTextField patientField, doctorField, dateField, timeField;
    private JTable appointmentTable;
    private DefaultTableModel tableModel;
    private List<Appointment> appointments = new ArrayList<>();

    public HospitalAppointmentGUI() {
        setTitle("Hospital Appointment Management");
        setSize(800, 520);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(10, 10));
        createMenuBar();

        JLabel title = new JLabel("HOSPITAL APPOINTMENT MANAGEMENT",
            SwingConstants.CENTER);
        title.setFont(new Font("SansSerif", Font.BOLD, 22));
        add(title, BorderLayout.NORTH);

        JPanel formPanel = new JPanel(new GridLayout(4, 2, 10, 10));
        formPanel.setBorder(BorderFactory.createTitledBorder("Appointment Details"));
        patientField = new JTextField();
        doctorField = new JTextField();
        dateField = new JTextField();
        timeField = new JTextField();

        formPanel.add(new JLabel("Patient Name:"));
        formPanel.add(patientField);
        formPanel.add(new JLabel("Doctor:"));
        formPanel.add(doctorField);
```

```
formPanel.add(new JLabel("Date (DD-MM-YYYY):"));
formPanel.add(dateField);
```

SOURCE CODE (CONTINUED)

```
formPanel.add(new JLabel("Time:"));
formPanel.add(timeField);

JButton bookButton = new JButton("Book");
JButton updateButton = new JButton("Update");
JButton cancelButton = new JButton("Cancel");
JButton clearButton = new JButton("Clear");

JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 5));
buttonPanel.add(bookButton);
buttonPanel.add(updateButton);
buttonPanel.add(cancelButton);
buttonPanel.add(clearButton);

JPanel topPanel = new JPanel(new BorderLayout(8, 8));
topPanel.add(formPanel, BorderLayout.CENTER);
topPanel.add(buttonPanel, BorderLayout.SOUTH);
add(topPanel, BorderLayout.CENTER);

tableModel = new DefaultTableModel(
    new String[]{"Patient", "Doctor", "Date", "Time", "Status"}, 0);
appointmentTable = new JTable(tableModel);
appointmentTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
add(new JScrollPane(appointmentTable), BorderLayout.SOUTH);

bookButton.addActionListener(e -> bookAppointment());
updateButton.addActionListener(e -> updateAppointment());
cancelButton.addActionListener(e -> cancelAppointment());
clearButton.addActionListener(e -> clearFields());

appointmentTable.getSelectionModel().addListSelectionListener(e -> {
    int row = appointmentTable.getSelectedRow();
    if (row >= 0) {
        patientField.setText(tableModel.getValueAt(row, 0).toString());
        doctorField.setText(tableModel.getValueAt(row, 1).toString());
        dateField.setText(tableModel.getValueAt(row, 2).toString());
        timeField.setText(tableModel.getValueAt(row, 3).toString());
    }
});

setVisible(true);
}

private void bookAppointment() {
    if (!validateInput()) return;
    Appointment a = new Appointment(
        patientField.getText().trim(),
        doctorField.getText().trim(),
        dateField.getText().trim(),
        timeField.getText().trim(),
        "Confirmed");
    appointments.add(a);
    refreshTable();
    JOptionPane.showMessageDialog(this,
        "Appointment confirmed successfully.",
        "Confirmation", JOptionPane.INFORMATION_MESSAGE);
    clearFields();
}

private void updateAppointment() {
    int row = appointmentTable.getSelectedRow();
    if (row < 0) {
        JOptionPane.showMessageDialog(this,
            "Select an appointment to update.",
            "Warning", JOptionPane.WARNING_MESSAGE);
        return;
    }
    if (!validateInput()) return;

    Appointment a = appointments.get(row);
```

SOURCE CODE (CONTINUED)

```
a.patient = patientField.getText().trim();
a.doctor = doctorField.getText().trim();
a.date = dateField.getText().trim();
a.time = timeField.getText().trim();
refreshTable();

JOptionPane.showMessageDialog(this,
    "Appointment updated successfully.",
    "Update", JOptionPane.INFORMATION_MESSAGE);
clearFields();
}

private void cancelAppointment() {
    int row = appointmentTable.getSelectedRow();
    if (row < 0) {
        JOptionPane.showMessageDialog(this,
            "Select an appointment to cancel.",
            "Warning", JOptionPane.WARNING_MESSAGE);
        return;
    }

    int choice = JOptionPane.showConfirmDialog(this,
        "Cancel selected appointment?",
        "Confirm Cancellation",
        JOptionPane.YES_NO_OPTION);

    if (choice == JOptionPane.YES_OPTION) {
        appointments.remove(row);
        refreshTable();
        JOptionPane.showMessageDialog(this,
            "Appointment cancelled successfully.",
            "Cancellation", JOptionPane.INFORMATION_MESSAGE);
        clearFields();
    }
}

private boolean validateInput() {
    if (patientField.getText().trim().isEmpty() ||
        doctorField.getText().trim().isEmpty() ||
        dateField.getText().trim().isEmpty() ||
        timeField.getText().trim().isEmpty()) {
        JOptionPane.showMessageDialog(this,
            "All appointment fields are required.",
            "Validation Error", JOptionPane.ERROR_MESSAGE);
        return false;
    }
    return true;
}

private void refreshTable() {
    tableModel.setRowCount(0);
    for (Appointment a : appointments) {
        tableModel.addRow(new Object[]{
            a.patient, a.doctor, a.date, a.time, a.status
        });
    }
}

private void clearFields() {
    patientField.setText("");
    doctorField.setText("");
    dateField.setText("");
    timeField.setText("");
    appointmentTable.clearSelection();
}

private void createMenuBar() {
    JMenuBar menuBar = new JMenuBar();
    JMenu fileMenu = new JMenu("File");
    JMenuItem exitItem = new JMenuItem("Exit");
```

SOURCE CODE (CONTINUED)

```
exitItem.addActionListener(e -> System.exit(0));
fileMenu.add(exitItem);

JMenu appointmentMenu = new JMenu("Appointments");
JMenuItem refreshItem = new JMenuItem("Refresh");
refreshItem.addActionListener(e -> refreshTable());
appointmentMenu.add(refreshItem);

menuBar.add(fileMenu);
menuBar.add(appointmentMenu);
setJMenuBar(menuBar);
}

static class Appointment {
    String patient, doctor, date, time, status;

    Appointment(String patient, String doctor,
                 String date, String time, String status) {
        this.patient = patient;
        this.doctor = doctor;
        this.date = date;
        this.time = time;
        this.status = status;
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(HospitalAppointmentGUI::new);
}
}
```